

Universitatea Transilvania din Braşov
Facultatea de Matematică şi Informatică
Specializarea: Informatică Aplicată

Studiu Preliminar (Faza 0)
Antrenare ViT-Tiny pe CIFAR-10
Cuantizare FP16, FP8 E4M3FN
Proof of Concept

Student:

Tudose Alexandru
Grupa 10LF333
An III, Informatică Aplicată

Coordonator:

Conf. dr. ing. Honorius Gâlmeanu
Informatică Aplicată

Cuprins

1	Introducere	2
2	Arhitectura Modelului și Dataset	2
2.1	Vision Transformer — ViT-Tiny	2
2.2	CIFAR-10	2
3	Design Experimental	3
3.1	Matricea experimentelor	3
3.2	Configurație antrenare	3
3.3	Hardware	3
4	Rezultate Antrenare	4
4.1	Comparație globală	4
4.2	Efectul augmentării	5
4.3	Efectul preciziei numerice	6
4.4	Curbe de loss	6
4.5	Timing	7
5	Cuantizare Post-Training FP8	7
5.1	Metodologie	7
5.2	Rezultate	8
5.3	Discuție	8
6	Monitorizare Hardware	9
7	Limitări ale Studiului Preliminar	9
8	Concluzii și Lecții Învățate	10
8.1	Concluzii	10
8.2	Lecții pentru fazele ulterioare	10

1 Introducere

Acest raport documentează **studiul preliminar** (Faza 0) al lucrării de licență, care a servit drept *proof of concept* pentru viabilitatea cuantizării numerice pe Vision Transformers. Experimentele au fost concepute pentru a valida pipeline-ul experimental și a demonstra fezabilitatea cuantizării înainte de trecerea la evaluarea pe ImageNet cu model pretrained.

Obiective:

1. Antrenarea unui ViT-Tiny pe CIFAR-10 cu diferite configurații de precizie (FP32 vs. FP16) și augmentare (basic vs. extended).
2. Evaluarea impactului fiecărui factor asupra acurateții, generalizării și timpului de antrenare.
3. Testarea cuantizării post-training FP8 (`float8_e4m3fn`) pe cel mai bun model antrenat.

Statut: Studiul preliminar este **finalizat**. Concluziile sale au informat direcția fazelor ulterioare (1–3) ale lucrării. A fost identificat faptul că abordarea FP8 nativă (`torch.float8_e4m3fn`) este problematică, ceea ce a condus la adoptarea cuantizării INT8 cu scalare liniară în fazele principale.

Notă: Acest raport este inclus în lucrarea de licență ca *studiu preliminar / test funcțional*. Rezultatele principale ale lucrării provin din Fazele 1–3 (evaluare pe ImageNet/ImageNette cu model pretrained).

2 Arhitectura Modelului și Dataset

2.1 Vision Transformer — ViT-Tiny

Modelul utilizat este `vit_tiny_patch16_224` din biblioteca `timm` [4], cu 5.7 milioane de parametri.

Tabela 1: Parametrii arhitecturali ai ViT-Tiny

Proprietate	Valoare
Parametri totali	5.7 milioane
Patch size	16×16 pixeli
Input resolution	224×224
Embedding dim	192
Număr blocuri	12
Număr capete (MHA)	3
MLP ratio	4 (hidden dim = 768)

2.2 CIFAR-10

CIFAR-10 conține 60 000 imagini de 32×32 pixeli, distribuite în 10 clase. Imaginile sunt redimensionate la 224×224 pentru compatibilitate cu ViT. Setul este împărțit în 50 000 de imagini de antrenare și 10 000 de imagini de validare.

Limitare: CIFAR-10 este un dataset simplu și mic, insuficient pentru a trage concluzii despre comportamentul cuantizării pe sarcini complexe. Rezultatele au fost utilizate exclusiv ca validare a pipeline-ului experimental.

3 Design Experimental

3.1 Matricea experimentelor

Au fost realizate 4 experimente combinând 2 factori:

Tabela 2: Matricea experimentelor 2×2

	Basic augmentation	Extended augmentation
FP32	BaseFP32	AugmFP32
FP16 (AMP)	BaseFP16	AugmFP16

Augmentare basic: Random horizontal flip + normalizare standard.

Augmentare extended: Random crop, horizontal flip, color jitter, random erasing, CutMix/MixUp, label smoothing.

3.2 Configurație antrenare

Tabela 3: Hiperparametri comuni

Parametru	Valoare
Epoci	50
Batch size	64
Optimizer	AdamW
Learning rate	3×10^{-4}
Scheduler	Cosine annealing (warm restarts)
Inițializare ponderi	Aleatorie (pretrained=False)
Dataset antrenare	CIFAR-10
Device	Apple M4 MPS
PyTorch	2.9.1

3.3 Hardware

Toate experimentele au fost executate pe MacBook Air cu Apple M4, 16 GB RAM, utilizând backend-ul MPS (Metal Performance Shaders) din PyTorch.

4 Rezultate Antrenare

4.1 Comparație globală

Tabela 4: Rezultate finale — toate experimentele

Experiment	Best Val Acc	Final Train Acc	Overfit Gap	Conv. Epoch	Timp (h)
BaseFP32	78.99%	97.29%	18.37 pp	43	7.26
BaseFP16	79.19%	97.43%	18.40 pp	35	7.45
AugmFP32	82.88%	86.80%	4.09 pp	45	7.38
AugmFP16	83.41%	85.67%	2.26 pp	48	7.09

Overfit Gap = Final Train Acc – Final Val Acc. Conv. Epoch = prima epocă ce atinge 99.5% din best val acc.

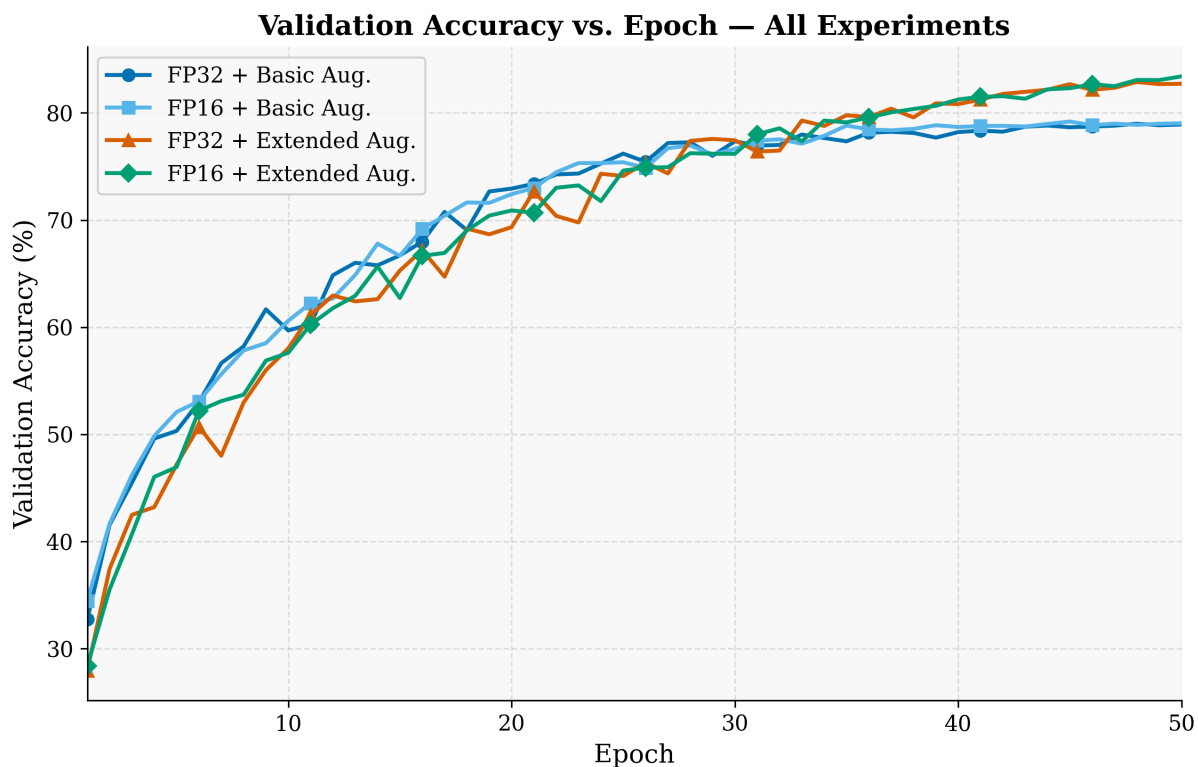


Figura 1: Curbele de acuratețe de validare pentru cele 4 experimente pe parcursul a 50 de epoci. Configurațiile cu augmentare extinsă (Augm*) ating o acuratețe semnificativ mai mare decât cele cu augmentare basic (Base*). Diferența între FP32 și FP16 este neglijabilă la același tip de augmentare.

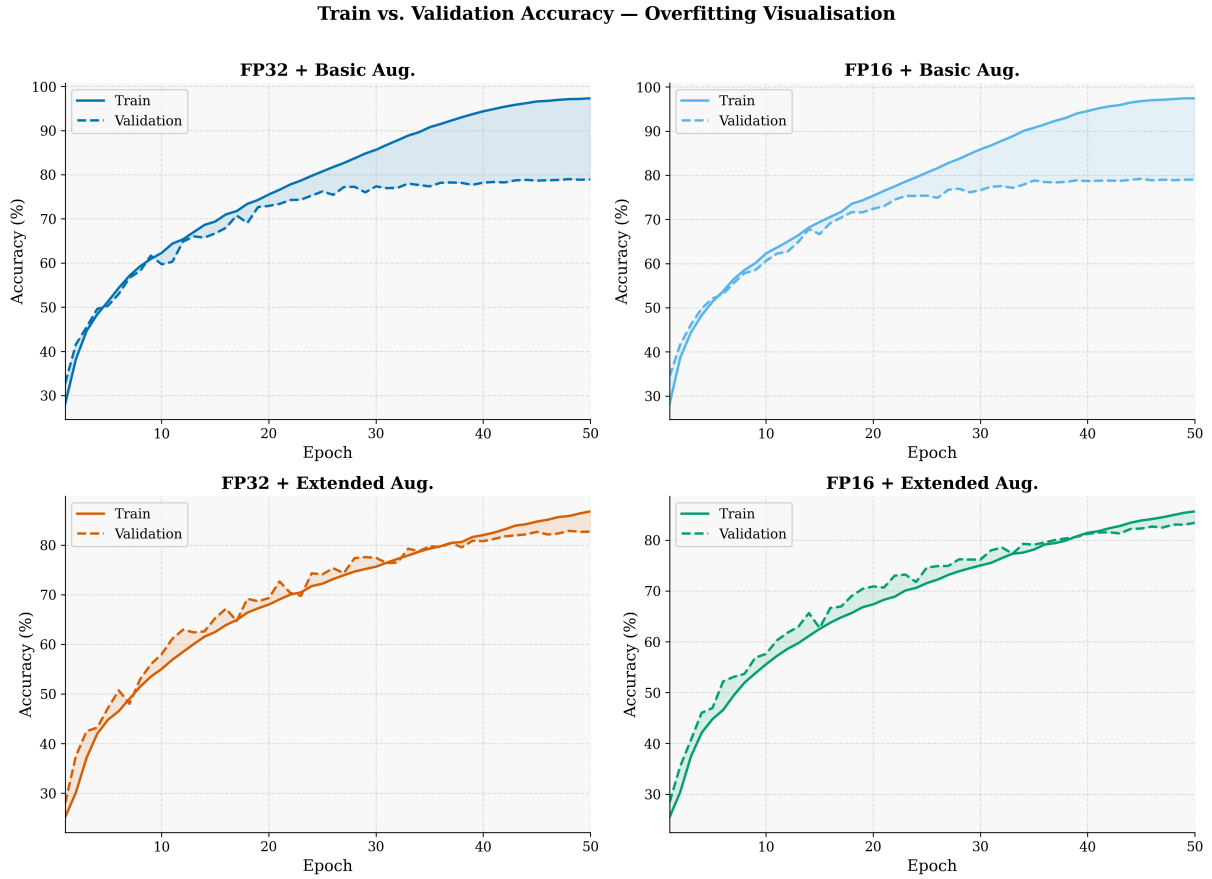


Figura 2: Grid 2×2 : acuratețe training vs. validare pentru fiecare experiment. Zona umbrită reprezintă gap-ul de overfitting. Experimentele Base* prezintă un gap masiv (≈ 18 pp), în timp ce Augm* reduce drastic overfitting-ul la 2–4 pp.

4.2 Efectul augmentării

Augmentarea extinsă este factorul dominant:

- **FP32:** Basic $\rightarrow$ Extended: +3.90 pp acuratețe
- **FP16:** Basic $\rightarrow$ Extended: +4.21 pp acuratețe
- Overfitting gap redus de la ≈ 18.4 pp la ≈ 2.3 –4.1 pp

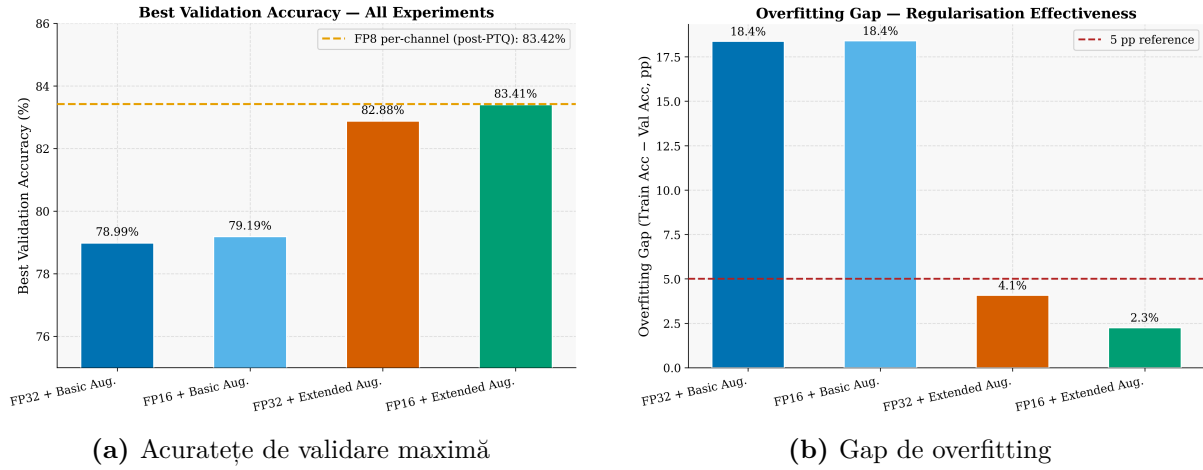


Figura 3: Comparație bar chart. (a) Augmentarea extinsă câștigă ≈ 4 pp acuratețe. (b) Overfitting-ul scade dramatic cu augmentare extinsă, de la 18.4 pp la 2.3 pp.

4.3 Efectul preciziei numerice

Diferența între FP32 și FP16 (AMP) este neglijabilă:

- Cu augmentare basic: FP16 este marginal mai bun cu +0.21 pp
- Cu augmentare extended: FP16 este mai bun cu +0.52 pp
- Timpii de antrenare sunt comparabili ($\pm 4\%$) — variațiile pot fi cauzate de condiții de sistem, nu de precizia numerică

Concluzie: Precizia numerică FP32 vs. FP16 nu are impact practic asupra acurateții pe CIFAR-10, confirmând robustețea ViT la reducerea preciziei.

4.4 Curbe de loss

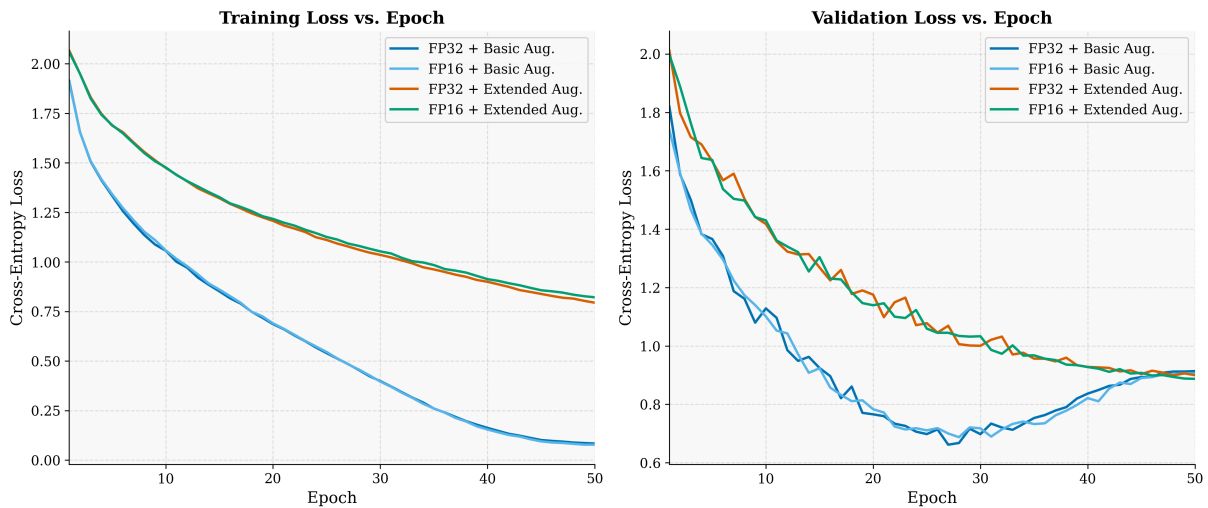


Figura 4: Curbele de loss (training și validare) pentru toate experimentele. Experimentele Base* converg la un loss de validare ridicat (≈ 0.9), cu un gap mare față de training loss (≈ 0.08), indicând overfitting sever. Experimentele Augm* mențin un gap mic între train și val loss.

4.5 Timing

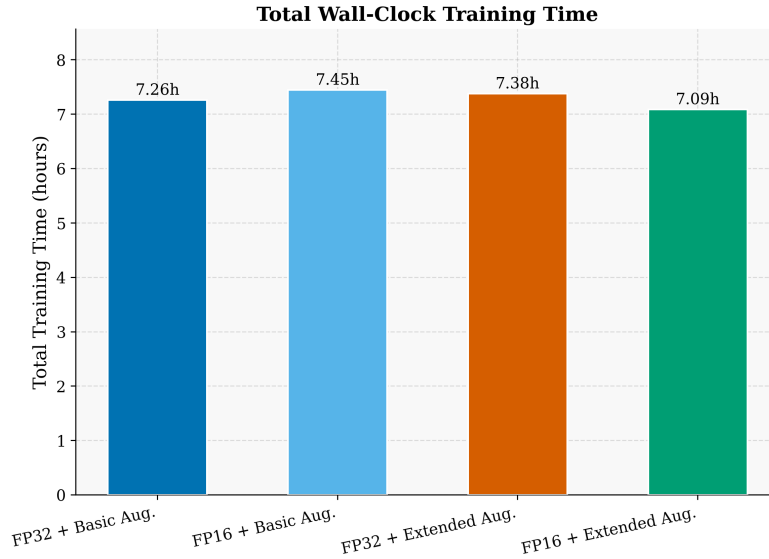


Figura 5: Timpul total de antrenare. Toate experimentele durează $\approx 7\text{--}7.5$ ore pe Apple M4 MPS, cu variații de $\pm 3\%$ între configurații.

Tabela 5: Timing per epocă

Experiment	Medie (sec)	Std (sec)
BaseFP32	522.90	± 16.45
BaseFP16	536.22	± 21.99
AugmFP32	531.33	± 27.68
AugmFP16	510.29	± 18.02

5 Cuantizare Post-Training FP8

5.1 Metodologie

Cel mai bun model antrenat (**AugmFP16**, acuratețe 83.41%) a fost supus cuantizării post-training la `float8_e4m3fn` la două granularități:

1. **Per-tensor:** Un singur factor de scalare per matrice de ponderi.

$$s = \frac{\text{FP8_MAX}}{\max(|W|)}, \quad \hat{W} = \text{clamp}(W \cdot s, -448, 448).to(\text{fp8}).to(\text{dtype})/s \quad (1)$$

2. **Per-channel:** Un factor de scalare per rând (dim 0) al matricii.

Notă importantă: Această abordare (cast direct la `float8_e4m3fn`) a fost ulterior identificată drept *problematică* de către coordonator, deoarece nu toate valorile FP32 sunt reprezentabile în formatul FP8. Fazele ulterioare (1–3) au adoptat cuantizarea INT8 cu scalare liniară, care garantează o mapare bijectivă la 256 valori discrete.

5.2 Rezultate

Tabela 6: Rezultate cuantizare FP8 pe modelul AugmFP16

Metodă	Acuratețe	Degradare	Loss
FP16 (original)	83.43%	—	0.5572
FP8 per-tensor	83.41%	+0.02 pp	0.5581
FP8 per-channel	83.42%	+0.01 pp	0.5576

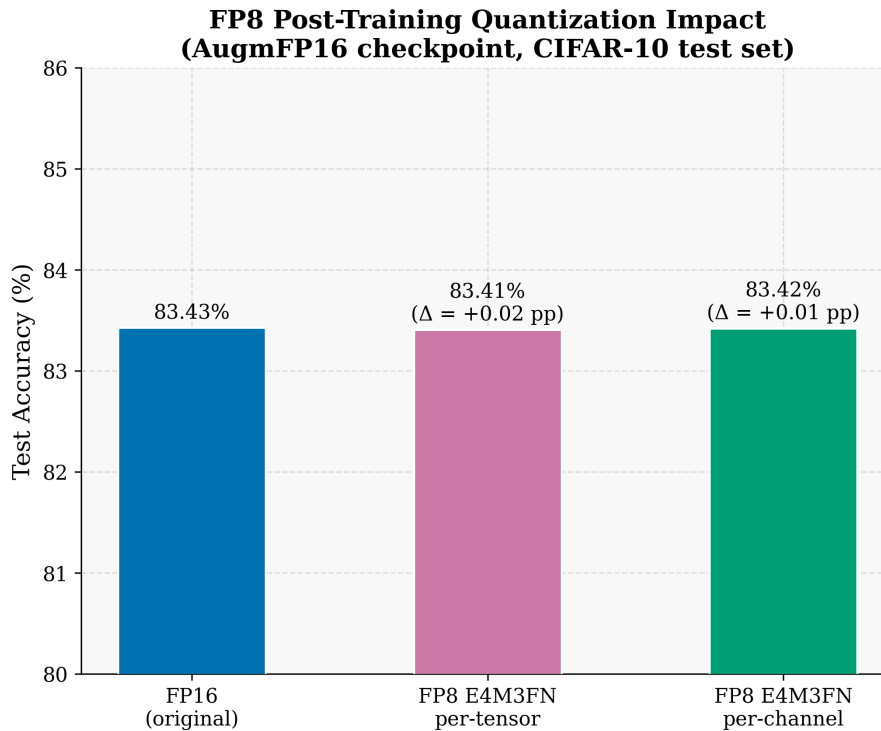


Figura 6: Impactul cuantizării FP8 asupra acurateții. Degradarea este esențialmente zero (< 0.02 pp), demonstrând viabilitatea cuantizării pe 8 biți pe acest model și dataset.

5.3 Discuție

Degradarea de +0.02 pp (per-tensor) și +0.01 pp (per-channel) este neglijabilă și se încadrează în variabilitatea numerică. Acest rezultat a confirmat ipoteza inițială: ViT-Tiny este robust la cuantizarea pe 8 biți.

Totuși, aceste rezultate trebuie interpretate cu prudență:

- CIFAR-10 este un dataset simplu — degradarea pe sarcini mai complexe poate fi semnificativ mai mare.
- Metoda de cuantizare FP8 (`float8_e4m3fn` direct) nu oferă aceeași garanție de mapare bijectivă ca INT8 cu scalare liniară.
- Toate ponderile au fost cuantizate, inclusiv LayerNorm și head — în fazele ulterioare, acestea sunt excluse.

6 Monitorizare Hardware

Tabela 7: Utilizare hardware în timpul antrenării

Experiment	CPU Avg	CPU Max	RAM Avg	RAM Max
BaseFP32	2.6%	54.3%	83.6%	89.3%
BaseFP16	2.6%	58.9%	81.6%	86.4%
AugmFP32	5.2%	59.1%	86.0%	90.8%
AugmFP16	4.8%	62.6%	81.6%	86.8%

Observații:

- CPU-ul este utilizat minimal ($< 5\%$ medie), indicând o sarcină GPU-bound pe MPS.
- Augmentarea extinsă crește utilizarea CPU de $\approx 2\times$ (preprocessing suplimentar).
- Memoria RAM este saturată (81–91%), sugerând că batch-ul de 64 este aproape de limita celor 16 GB.
- Niciun experiment nu a declanșat thermal throttling.

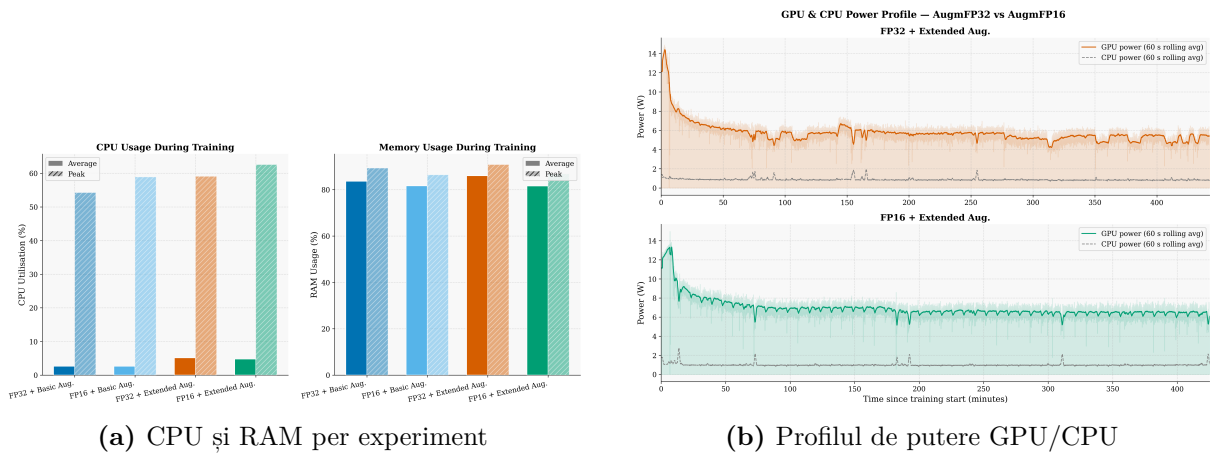


Figura 7: (a) Utilizare CPU și RAM medie/maximă. (b) Profilul de consum energetic în timp pentru AugmFP32 vs. AugmFP16 — profiluri similare, confirmând că FP16 nu introduce overhead suplimentar pe Apple M4.

7 Limitări ale Studiului Preliminar

1. **Dataset:** CIFAR-10 este prea simplu și mic (60 000 imagini, 10 clase) pentru a trage concluzii valide despre comportamentul cuantizării pe sarcini reale. Rezultatele nu se generalizează direct la ImageNet.
2. **Metoda FP8:** Cast-ul direct la `float8_e4m3fn` nu garantează o mapare bijectivă — anumite valori FP32 nu sunt reprezentabile în FP8 E4M3FN. Fazele ulterioare adoptă INT8 cu scalare liniară.
3. **Cuantizare neselectivă:** Toate ponderile au fost cuantizate, inclusiv LayerNorm și classification head, care sunt sensibile la erori numerice. Abordarea corectă (Faza

2+) exclude aceste straturi.

4. **Fine-tuning, nu inferență pură:** Studiul preliminar antrenează modelul pe CIFAR-10, deci evaluează cuantizarea *după fine-tuning*. Fazele principale evaluează cuantizarea pe un model pretrained *fără nicio re-antrenare* (post-training quantization).
5. **Hardware:** Experimentele pe M4 nu evaluează speedup-ul real al cuantizării pe hardware cu suport nativ (NVIDIA Tensor Cores).

8 Concluzii și Lecții Învățate

8.1 Concluzii

1. **Augmentarea domină:** Augmentarea extinsă produce un câștig de +3.9–4.2 pp acuratețe, reducând overfitting-ul de la 18.4 pp la 2.3 pp. Acesta este factorul cu cel mai mare impact asupra performanței.
2. **FP32 vs. FP16 — diferență neglijabilă:** Precizia numerică nu afectează semnificativ acuratețea ($\Delta < 0.5$ pp), confirmând robustețea ViT la `float16`.
3. **Configurația optimă: AugmFP16** — acuratețe 83.41%, overfitting gap 2.26 pp, timp minim (7.09 h).
4. **FP8 viabil ca proof of concept:** Degradarea < 0.02 pp pe CIFAR-10, dar metoda FP8 nativă prezintă limitări teoretice.

8.2 Lecții pentru fazele ulterioare

- Trecerea de la CIFAR-10 la **ImageNet2-320** (proxy ImageNet) pentru evaluări mai relevante.
- Adoptarea **INT8 cu scalare liniară** în locul FP8 nativ — mapare bijectivă garantată.
- **Cuantizare selectivă:** excluderea LayerNorm, cls_token, pos_embed și head.
- Evaluare în regim **post-training quantization** (fără re-antrenare), pe model pretrained.
- Analiza **per-channel vs. per-tensor** și **sensitivity analysis** per strat.

Bibliografie

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *ICLR*, 2021. [arXiv:2010.11929](https://arxiv.org/abs/2010.11929).
- [2] H. Touvron, M. Cord, M. Douze et al., “Training data-efficient image transformers & distillation through attention,” *ICML*, 2021. [arXiv:2012.12877](https://arxiv.org/abs/2012.12877).
- [3] P. Micikevicius, D. Stosic, N. Burgess et al., “FP8 Formats for Deep Learning,” 2022. [arXiv:2209.05433](https://arxiv.org/abs/2209.05433).
- [4] R. Wightman, “PyTorch Image Models (timm),” 2019. <https://github.com/rwightman/pytorch-image-models>.

- [5] A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” Technical Report, University of Toronto, 2009.